

ICS Exercise Week 6 Solutions

2026-04-10

Problem 1

1. Two child processes are created.
 2. The parent can observe exit statuses `100` and `101`.
 3. No. The parent does not have to observe them in a fixed order, because the two children run concurrently and may terminate in either order.
 4. The loop with `waitpid(-1, &status, 0)` repeatedly reaps whichever child terminates next, until no children remain. Waiting only once would reap at most one child (*the last forked child process*) and could leave another zombie behind.
-

Problem 2

```

void run_cmd(char *cmd, char **argv, int bg) {
    pid_t pid;
    int status;

    if ((pid = fork()) < 0) {
        /* Handle fork error */
        unix_error("fork error");
    }
    else if (pid == 0) {
        /* child */
        if (execve(cmd, argv, environ) < 0) {
            printf("%s: Command not found.\n", cmd);
            exit(127);
        }
    }

    /* parent */
    if (!bg) {
        if (waitpid(pid, &status, 0) < 0) {
            unix_error("waitpid error");
        }
    } else {
        printf("[%d] %s &\n", (int)pid, cmd);
    }
}

```

Problem 3

1. The most likely `errno` value is `EINTR`.
2. This happens because `waitpid` can be interrupted by a signal before the target child changes state.
3. Correct rewrite:

```
int status;
pid_t ret;

while ((ret = waitpid(pid, &status, 0)) < 0) {
    if (errno != EINTR) {
        printf("waitpid failed\n");
        exit(1);
    }
}
```

This retries only when the failure was caused by interruption.

Problem 4

1. The foreground job's process group `5000` should receive the signal.
 2. The signal is `SIGINT`.
 3. Every process in process group `5000` receives `SIGTSTP`.
 4. A negative PID in `kill` tells the kernel to interpret the absolute value as a process group ID rather than a single process ID.
-

Problem 5

```

#include <errno.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <unistd.h>

/* Helper functions provided by the CS:APP library */
int open_listenfd(int port);
void wait_disconnect(int fd);

void handler(int sig) {
    /* (a) Complete the handler: save errno, write message, and reset alarm */
    int old_errno = errno;          // Save errno
    write(1, "HEARTBEAT\n", 10); // Async-signal-safe I/O
    alarm(1);                       // Schedule next heartbeat
    errno = old_errno;              // Restore errno
}

int main(void) {
    int listenfd, connfd;
    sigset_t mask, prev;

    /* (b) Initialize signal mask to block SIGALRM */
    sigemptyset(&mask);
    sigaddset(&mask, SIGALRM);

    /* (c) Setup handler and initial alarm */
    signal(SIGALRM, handler);
    alarm(1);

    listenfd = open_listenfd(1234);
    while (1) {
        connfd = accept(listenfd, NULL, NULL);

        /* Enter Redirect Mode */
        /* Block signals before modifying file descriptors */
        /* (d) Start Critical Section */
        sigprocmask(SIG_BLOCK, &mask, &prev);
        int stdout_backup = dup(1);
        dup2(connfd, 1);
    }
}

```

```
    /* (e) End Critical Section */
    sigprocmask(SIG_SETMASK, &prev, NULL);

    /* Wait for client to disconnect */
    wait_disconnect(connfd);

    /* Transition back to "Normal Mode" */
    /* Block signals again during the sensitive restoration process */
    /* (f) Start Critical Section */
    sigprocmask(SIG_BLOCK, &mask, &prev);
    dup2(stdout_backup, 1);
    close(connfd);
    close(stdout_backup);
    /* (g) End Critical Section */
    sigprocmask(SIG_SETMASK, &prev, NULL);
}
return 0;
}
```